

DATA203 Foundational Python (Prof. Maull) / Spring 2026 / HW2

Points Possible	Due Date	Time Commitment (estimated)
25	Friday, April 3 @ midnight	up to 15 hours

- **GRADING:** Grading will be aligned with the completeness of the objectives.
- **INDEPENDENT WORK:** Copying, cheating, plagiarism and academic dishonesty *are not tolerated* by University or course policy. Please see the syllabus for the full departmental and University statement on the academic code of honor.

OBJECTIVES

- Practice writing functions and loading data from files.
- More practice writing functions, explaining what they do and writing DocString documentation for a function.
- Load a JSON data file and write functions to manipulate data.
- BONUS: Do more advanced things with your prior functions and data outputs.

WHAT TO TURN IN

You will enjoy the highest benefits of the starter notebook if you clone the HW Github repository from your Jupyter Hub terminal with the command:

```
git pull https://github.com/kmhuads/s26_data203.git
```

This will ensure you have the most updated files and starter notebook.

Once you have cloned the repository, you can edit the starter notebook with your solution code.

When you are done with your work, it will be best to zip your hw2 folder and all sub-folders with the terminal command (one level outside your notebook folder):

```
zip -r data203_hw2_maull.zip ./hw2
```

This will produce the file with all necessary supporting files (notebooks, data output, etc.) then download it from the Jupyter Hub to your local machine, then upload the .zip to Teams.

If are confused on how to do this, please ask, or visit one of the many tutorials on the basics of using zip in Linux.

If you choose not to use the provided notebook, you will still need to turn in a .ipynb Jupyter Notebook and corresponding files according to the instructions in this homework.

ASSIGNMENT TASKS

(25%) Practice writing functions and loading data from files.

We can all use a little more practice writing functions in Python, so here we go.

In the data/ folder, you will see two files data/1_firstnames.txt and data/1_lastnames.txt.

These files have a thousand names each, you will use those files to perform the following tasks.

\$ Task: 1.0 Write a function to load each file into a dictionary based on name length.

This should be very straightforward, but you will need to have the following:

1. In the starter notebook, you will see a dictionary called names. You will also notice that there are two keys in it already: firstnames and lastnames.

2. You will need to load each file of 1000 names and put them into the appropriate keys (first names will go into `names['firstnames']` last names will go into `names['lastnames']`).
3. *However* there is a twist. You must put the names into a list using keys based on length. That is to say, each of `names['firstnames']` and `names['lastnames']` will have numeric keys themselves indicating the *length* of the name. The *value* of the *key* will be a list containing the list of names of that length.

For example, if you find a name “Alexa”, which is 5 letters in length, you will append it to the list of the key 5 in `names['firstnames']`. The dictionary with key 5 will have the value `['Alexa']` if Alexa is the first 5 letter name found. When another 5 letter name is found, say Daryl the key 5 will contain the list `['Alexa', 'Daryl']`.

Concretely, the names dictionary might look something like this:

```
{
  'firstnames':
    3: ['Tim', 'Tom'],
    4: ['Greg', 'Fred'],
    5: ['Alexa', 'Daryl']
    ...
  'lastnames':
    5: ['Smith', 'Herle']
    ...
}
```

4. You are *required* to write the function `make_names_dict(f_first, f_last)` which **returns** the dictionary of names based on data in the files `f_first` and `f_last`.

Here are some other bits of information you will find useful:

function NAME: `make_names_dict(f_first, f_last)`

function INPUT

- `f_first`: a string filename with firstnames
- `f_last`: a string filename with lastnames

function OUTPUT

- the dictionary object of names in each file with keys "firstnames" and "lastnames"
- the keys "firstnames" and "lastnames" keys will have keys themselves which are numbers indicating the length of the name
- each of the numbered keys will have a list of the names of that length

Code Example:

```
make_names_dict("file001.txt", "file002.txt")
```

Output:

```
{
  'firstnames':
    3: ['Tim', 'Tom'],
    4: ['Greg', 'Fred'],
    5: ['Alexa', 'Daryl']
    ...
  'lastnames':
    5: ['Smith', 'Herle']
    ...
}
```

Consult the starter notebook for additional hints.

\$ Task: 1.1 Use the json module to dump the dictionary of names from the first part into a file called "data/names_dict.json".

Study the [json module](#) and learn to use the function `dump()`.

\$ Task: 1.3 Write a function `name_gen()` which takes three parameters, `names`, `first_len` and `last_len` for length of firstname and lastname, respectively.

The function will use the `names` parameter as the source of the names using the `make_names_dict()` function from the first part.

function NAME: `name_gen(names, first_len, last_len)`

function INPUT

- *names* : dictionary of first and last names
- *first_len*: length of firstname
- *last_len*: length of lastname

function OUTPUT

- a string representing the name of length given by the *first_len* and *last_len* parameters.

Code Example:

```
name_gen(names, 5, 7)
```

Output:

Grace Simpson

(25%) More practice writing functions, explaining what they do and writing DocString documentation for a function.

We learned in lecture that documentation for Python built-ins can be accessed with `help()`. We also learned that this same tool can be used on modules and packages that we import.

But *how* did this documentation get created?

To answer that question you will need to study Python DocStrings here:

- [Python DocStrings coding standards by Google](#)

In the notebook are some scaffolding code. You will need to study it and use it in the solutions being asked.

\$ Task: 2.0 Practice explaining code.

Use the code in the first cell of the provided notebook and answer the question below:

1. Explain in your own words what function `mystery_function` in the cell does.

Your explanation should include the description of the inputs, and you will need to review the Python [random](#) libraries and specifically `randint()` for details of the function.

2. Write DocStrings documentation for the function. Use the style provided by the Google coding standards link already provided.

\$ Task: 2.1 Write a function which uses the output of `mystery_function()`.

We are now going to use the output of the mystery function as a *model*, which is typical of computation in general – we are able to use the *models* we make in the machine as an *approximate* representation of reality, which is what a model actually is! We must, however, realize that some models are *better* approximations than others and we are always on the hunt for the *best* models we can develop.

In our case, the mystery function will model a concert hall.

Each item in the return object of the function represents a physical row in the concert hall. So, for example, the *0th* object (a list in our case) is the first row of the hall (closest to the stage), the *1st* object (the next list) represents the next row and so on.

Next, each item in the object (list item) represents a seat. If there is a 1 in an item, then that seat is occupied, if there is a 0 then it is not occupied.

You will write a function, which will count the number of attendees at a concert, given by the given test invocations of `mystery_function` in the starter notebook.

The name of the function will be `attendance_count()` and is specified below:

function NAME: `attendance_count`

function INPUT:

- `m`: model given by `mystery_function`

function OUTPUT:

- a count of the 1 values in the model `m`, which is a count of the attendance for that concert
- the return value will be a single integer

Code Example:

```
m = mystery_function(100, 100) # 100 rows, 100 seats per row
attendance_count(m) # return the number of 1s in m
```

Example Output:

120

Don't overthink this – your solution will be a very simple counter, and there are even some shortcuts, which you can use (HINT: explore the built-in `sum()`). There is even an elegant *one line solution* using `sum()` and `map()` together.

(50%) Load a JSON data file and write functions to manipulate data.

We talked about file I/O in lecture and now that we have an idea about how to go from Python dictionaries to JSON we are now going to put all of these things together.

Apropos for the season, we are going to pretend we are tax accountants, and we have a large number of clients to get to the finish line.

Specifically, we would like to take the remaining 250 clients and put them into priority groups – group “P1” we will call is the group of clients which will need to give high priority attention to. “P2” is intermediate priority and “P3” the low priority group.

We will decide which accounts belong to which groups as part of this part of the assignment.

You will need to look at the client JSON file at:

- `data/3_tax_clients.json`

Continue to study the starter notebook and follow the questions below.

§ Task: 3.1 Write a function `compute_taxes` which returns a tuple specified below.

function NAME: `compute_taxes`

function INPUT:

- `i`: the income
- `d`: the deductions
- `tp`: the taxes paid

function OUTPUT:

- a tuple containing the *taxable income*, *estimated taxes* and the *amount of taxes owed* (which is the difference between estimated taxes and taxes paid, `tp`)

Code Example:

```
# income=10000, deductions=3000, taxes_paid=1000

m = compute_taxes(10000, 3000, 1000)
print(m)
```

Example Output:

```
(7000, 700.0, -300)
# the tuple of taxable income, estimate taxes and taxes owed
```

You will find the following information important, so READ CAREFULLY:

- *taxable income* is the *income* minus *deductions*; for example if *i* is income and *d* the deductions, taxable $ti = i - d$
- *estimated taxes* will be calculated using this function provided in the starter notebook – do NOT alter or change the function:

```
def estimated_tax(ti):
    if ti < 100000:
        return ti * 0.10
    if ti < 200000:
        return ti * 0.15
    if ti < 300000:
        return ti * 0.20
    if ti < 400000:
        return ti * 0.25
    else:
        return ti * .30
```

\$ Task: 3.2 Write a function `assign_priority()` to put a customer in a priority group.

Your function will take a customer record from the `tax_returns.json` file and return a 1, 2 or 3 using the following scheme:

Priority 3: if the amount of taxes owed is negative (a refund, since taxes were *overpaid*)

Priority 2: if the amount of taxes owed is positive, but the *estimated taxes* is within 5% of the *taxable income* (the client underpaid, but not significantly)

Priority 1: if the amount of *estimated taxes* is more than +/- 15% of the *taxable income* in either direction (positive or negative), then the client significantly under or overpaid thus more attention may be necessary.

function NAME: `assign_priority`

function INPUT: - *r*: customer record

function OUTPUT:

- an integer containing the priority level

Code Example:

```
r = json.load(open("data/3_tax_clients.json"))[0] # first record sample only
p = assign_priority(r)
print(p)
```

Example Output:

```
1
# priority level 1
```

HINT: You will need to use `compute_taxes()` in `assign_priority()`.

\$ Task: 3.3 Assign a priority to ALL customers in `data/tax_clients.json`.

Load the file and run `assign_priority` to all customers.

You *can* write a function, but you do not have to – you can run the code in a cell and emit the data.

Your output, however, should look like this:

```
Customer 1: P1
Customer 2: P1
...
```

```
Customer 120: P3
...
Customer 250: P0
```

To get the output to look like this, consider using format strings and taking some inspiration from the code below:

```
customer_records = json.load(open("data/3_tax_clients.json"))
for r in customer_records:
    priority = assign_priority(r)

    print(f"Customer {r['cust_id']}: P{priority}")
```

(0%) BONUS: Do more advanced things with your prior functions and data outputs.

There are so many things we can now do with our name generators, tax prioritizers, etc. but there is only limited amounts of time.

If you want to earn some more points, or just try something more challenging attempt one or more of the options below.

You can earn several BONUS points *per* task.

\$ Task: B.1 Write a function that generates a list of 50 unique random names.

Use your `name_gen()` function repeatedly to generate a list of 50 random names like:

```
["Sam Grant", "Dahlia Underwood", ... , "Ade Opponbaah"]
```

- You will earn more points if you check for uniqueness.

\$ Task: B.2 Write a function that instead of using length, uses letters as input.

For example, `name_gen(names, 's', 'v')` will generate a name where the firstname starts with s and the lastname starts with v.

- You will earn the most points if you alter the way `make_names_dict()` works, so that it stores letters as keys instead of name length. Concretely, instead of numeric keys, you would have character keys:

```
{
    "firstnames":
        "a": ["angie", "anderson", ...]}

...

    "lastnames":

        ...

        "c": ["calhoun", "caldridge", ...]

    ...
}
```

You will find this greatly facilitates what is being asked in this bonus part.

\$ Task: B.3 Create a dictionary which contains the customer numbers and the priority group they belong to.

This takes the tax work a bit further and to get points here, you will need to create a dictionary structure which looks like this:

```
{
    1: [1, 7, 8, 21, ... , 123]
    2: [4, 5, 6, 9, ... , 211]
    3: [77, 121, 122, 179, ... , 244]}
}
```

- You will earn points by using your `assign_priority()` function, along with adding the `cust_id` to the dictionary key (1, 2 or 3) it belongs to based on the value of the function call.